Deliverable #2 Template

SE 3A04: Software Design III – Large System Design

Tutorial Number: T01
Group Number: G01
Group Members:

- Benjamin Bloomfield
- Vikram Chandar
- Trisha Panchal
- Yug Vashisth
- Hamza Nadeem

IMPORTANT NOTES

- Please document any non-standard notations that you may have used
  - *Rule of Thumb*: if you feel there is any doubt surrounding the meaning of your notations, document them
- Some diagrams may be difficult to fit into one page
  - Ensure that the text is readable when printed, or when viewed at 100% on a regular laptop-sized screen.
  - If you need to break a diagram onto multiple pages, please adopt a system of doing so and thoroughly explain how it can be reconnected from one page to the next; if you are unsure about this, please ask about it
- Please submit the latest version of Deliverable 1 with Deliverable 2
  - Indicate any changes you made.
- If you do <u>NOT</u> have a Division of Labour sheet, your deliverable will <u>NOT</u> be marked

1 Introduction

This section should provide an brief overview of the entire document.

1.1 Purpose

State the purpose and intended audience for the document.

This document is intended to provide a high-level overview of the SCEMAS system architecture design as well as considerations for various subsystems that are a part of SCEMAS.
The intended audience for this document is stakeholders of SCEMAS, including developers and testers. SCEMAS deliverable 1 would serve as a necessary document to aid in understanding this document.

1.2 System Description

Give a brief description of the system. This could be a paragraph or two to give some context to this document.

The SCEMAS system is designed to collect, process, analyze, and display environmental data from sensors placed throughout a city. Its main purpose is to serve as a monitoring system that evaluates and alerts users based on set environmental parameters. The system will evaluate sensor data against environmental level thresholds. When data exceeds thresholds, the system will send an alert for users to view and act on. The system will provide a private dashboard for city officials and a public API for public users.

1.3 Overview

Describe what the rest of the document contains and explain how the document is organised (e.g. "In Section 2, we discuss...in Section 3...").

In Section 2 of this document, an analysis class diagram is provided for SCEMAS. In Section 3, the document covers the architectural design of the system, including architecture choices, justifications, a structural architecture diagram, and design alternatives that were considered. Finally, in Section 4, the document provides Class Responsibility Collaboration Cards that help further detail the connections of each class from the analysis class diagram from Section 2.

2 Analysis Class Diagram

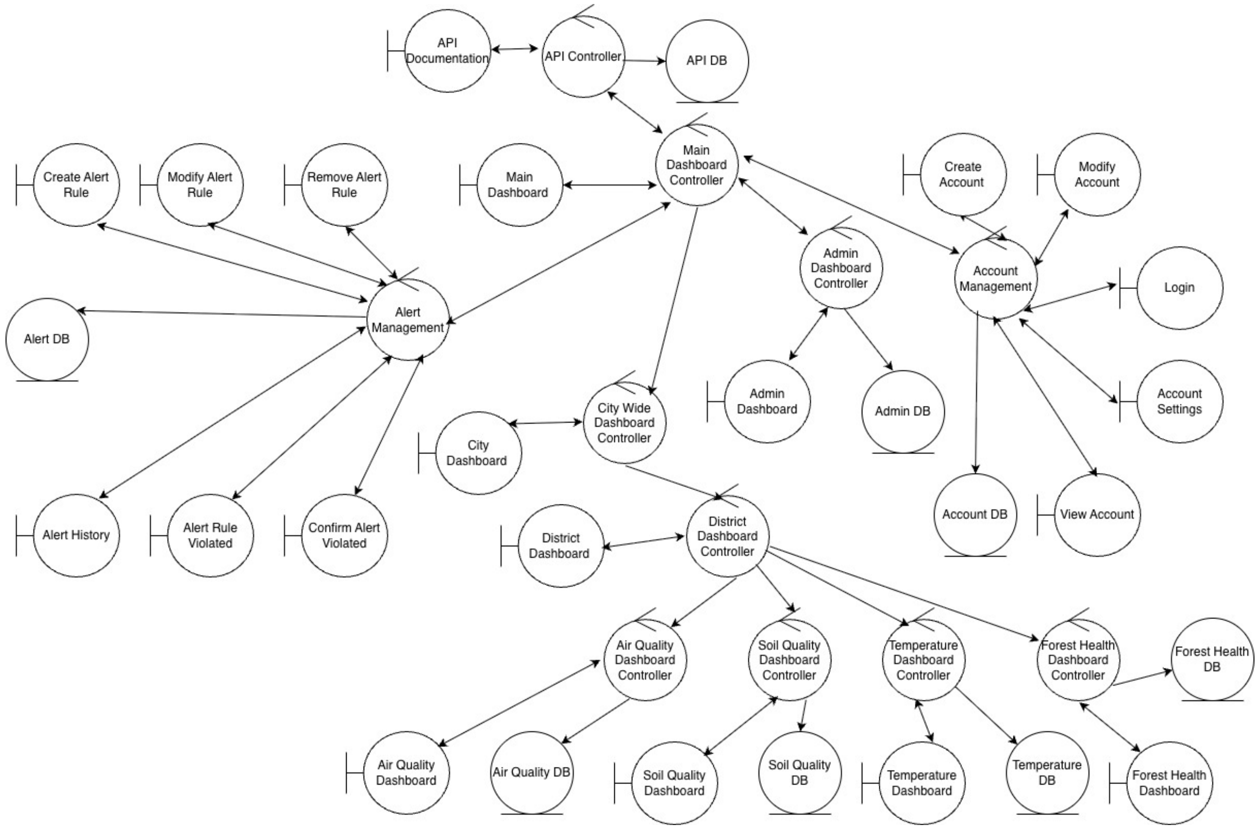


Figure 1: Analysis Class Diagram

3 Architectural Design

This section should provide an overview of the overall architectural design of your application. Your overall architecture should show the division of the system into subsystems with high cohesion and low coupling.

3.1 System Architecture

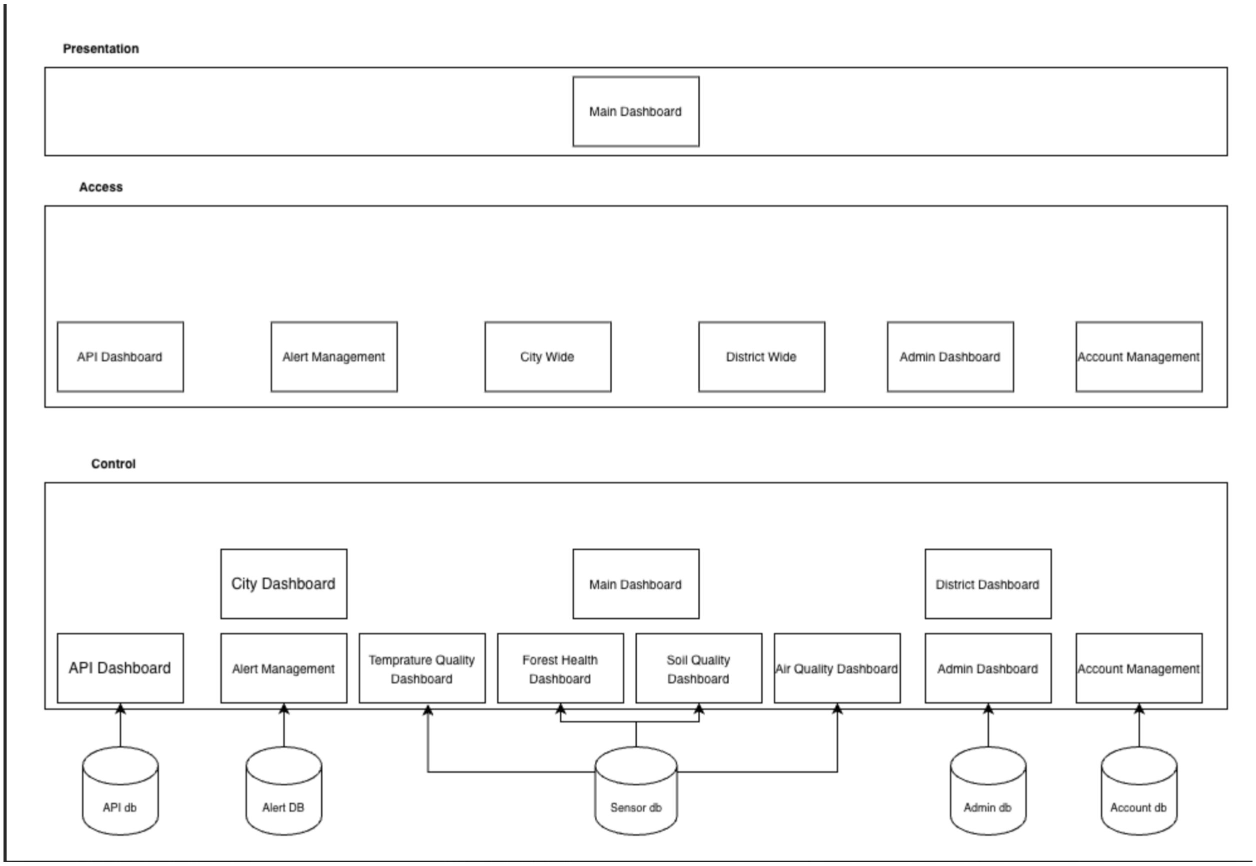


Figure 2: Structural Architecture Diagram for SCEMAS

- Identify and explain the overall architecture of your system
- Be sure to clearly state the name of the architecture you used (this is the name of the architectural pattern, not the name of your system)
- Provide the reasoning and justification of the choice of architecture
- Provide a structural architecture diagram showing the relationship among the subsystems (if appropriate)
- List any design alternatives you considered, but eliminated (and explain why you eliminated them)

3.1.1 Considered Design Alternatives

Model-View-Controller (MVC)
The MVC architectural pattern was considered because the system uses web dashboards to display environmental data. MVC is best used to organize interactive applications where a single central model supports multiple views, and user interactions are the main concern of the system. It is especially well suited for UI-based systems that focus on maintaining consistency between the interface and data. The SCEMAS software contains multiple complex components (telemetry ingestion, alert management, rule evaluation, etc.), which each have their own logic and control flow, with data changing and different periods and frequencies. As these components must be coordinated at a system level, is is challenging to maintain the structure with a PAC architecture. MVC assumes a centralized structure between the model, view, and controller, which

does not support the hierarchical structure that PAC offers.

Blackboard
The Blackboard architecture was considered since the system involves multiple subsystems that interact with a centralized data store. Blackboard is most appropriate for subsystems in which agents collaborate by contributing partial solutions to a problem stored in the blackboard. This architecture was eliminated since SCEMAS does not involve collaborative problem-solving among knowledge sources. Its subsystems do not compete, or contribute partial solutions. Instead, interactions with the data store are agent-driven, through read and writes. The data store therefore simply acts as passive storage. Since there is no need for agents to incrementally work on a shared solution state, the blackboard would simply introduce unnecessary coordination complexity.

Batch Sequential Architecture
The Batch Sequential Architecture is designed for systems that process data in discrete batches. Each processing stage must complete before the next stage starts, meaning that data flows linearly. SCEMAS, however, requires real-time (continuous) processing of environmental telemetry. Sensor data must be evaluated in real time to generate rule violations and environmental alerts without delay. Since the Batch Sequential architecture relies on periodic batch execution, it would introduce latency and prevent timely alert generation. Therefore, the Batch Sequential architecture was not selected.

Additional Data Flow Architectures
The Pipe-and-Filter and Process-Control architectural styles were not considered as design alternatives because they had not yet been introduced in the course lectures when the architectures were discussed.

3.2 Subsystems

The system has been divided into three primary subsystems:

1. Alert Management
2. Account Management
3. Dashboard
4. Public API

3.2.1 Alert Management Subsystem

The Alert Management subsystem is responsible for evaluating the environmental data against the predefined rules and generating alerts when the system detects a threshold violation. This may include environmental events such as wildfire risks, extreme heat conditions, or potential for natural disasters, such as hurricanes. The subsystem manages the life cycle of these alerts, including when they are created, confirmed, and resolved. It is also responsible for maintaining a history of all logged alerts and their current status. This isolation of concerns helps to ensure high-cohesion of alert functionalities, and maintain loose coupling with storage and presentation components.

This subsystem is responsible for interacting with the Repository and Dashboard subsystem. It needs to communicate with the Repository to retrieve the environmental data required for alert evaluation. Its interactions with the Dashboard are essential to provide real-time updates on the status of a particular alert. When a threshold violation is detected, the subsystem generates a corresponding alert event, which is uploaded to the dashboard for City Operators to view.

3.2.2 Account Management Subsystem

The Account Management subsystem is responsible for authenticating and authorizing the different users of the system and ensuring that role-based access control (RBAC) is sustained. It allows users to register, log in, and manage their account details. It also defines the different user roles (city operators, public users) and ensures that only those who have permission to access a specific functionality are able to do so. Separating these details into its own subsystem is essential to help enhance the overall security while ensuring that authentication operates independently.

Like the Alert Management subsystem, it also needs to collaborate with the Repository. This is needed to securely store user information and profile details.

3.2.3 Dashboard Subsystem

The dashboard subsystem is essential for providing an interface to display environmental data and alerts. It displays the real-time sensor readings, paleographical maps, sensor locations, and environmental metrics. This is fundamental for allowing city operators to monitor environmental conditions and make necessary responses.

The Dashboard subsystem interacts with the Alert Management subsystem to display current and historical alerts. It also communicates with the Repository to retrieve environmental data and metrics. The Dashboard subsystem follows the Presentation Abstraction Control (PAC) architecture pattern to separate user interface elements, interaction logic, and data models.

3.2.4 Public API Subsystem

The Public API subsystem provides controlled access to environmental data and alert information through its REST endpoints. Third-party developers and public users can request to retrieve environmental data and relevant alert information generated by the system.

This subsystem primarily communicates with the Alert Management and Repository subsystems. To satisfy client requests, it retrieves environmental and alert data from the Repositories. It also collects alert data generated by the Alert Management subsystem to ensure that alerts stay up to date when the public wants to view them. By separating how this data is accessed externally, with a separate subsystem, the architecture keeps internal system components loosely coupled with external users.

4 Class Responsibility Collaboration (CRC) Cards

This section should contain all of your CRC cards.

- Provide a CRC Card for each identified class
- Please use the format outlined in tutorial, i.e.,

Class Name: Main Dashboard Controller	
Responsibility:	**Collaborators:**
Knows Main Dashboard	Main Dashboard
Knows Account Management	Account Management
Knows Admin Dashboard Controller	Admin Dashboard Controller
Knows Alert Management	Alert Management
Knows API Controller	API Controller
Facilitates communication regarding overall dashboard structure	

Table 1: CRC Card for Alert Management Controller

Class Name: Alert Management Controller	
Responsibility:	**Collaborators:**
Knows Alert Database	Alert Database
Knows Create Alert Rule	Create Alert Rule
Knows Modify Alert Rule	Modify Alert Rule
Knows Remove Alert Rule	Remove Alert Rule
Knows Alert Rule Violated	Alert Rule Violated
Knows Confirm Alert Rule Violated	Confirm Alert Rule Violated
Knows Alert History	Alert History

Table 2: CRC Card for Alert Database Entity

Class Name: Alert Database (Entity)	
Responsibility:	**Collaborators:**
Knows Alert Management	Alert Management
Handles overall alert storage	

Table 3: CRC Card for Creating an Alert Rule

Class Name: Create Alert Rule (Boundary)	
Responsibility:	**Collaborators:**
Handles interfacing for creating new alert rules Collects and validates alert rule parameters Sends the creation request to the Alert Management Controller	Alert Management Controller

Table 4: CRC Card for Modifying an Alert Rule

Class Name: Modify Alert Rule (Boundary)	
Responsibility:	**Collaborators:**
Handles interfacing for updating alert rules Displays current rule information Validates rule modifications Sends the modification request to the Alert Management Controller	Alert Management Controller

Table 5: CRC Card for Removing an Alert Rule

Class Name: Remove Alert Rule (Boundary)	
Responsibility:	**Collaborators:**
Handles interfacing for deleting an alert rule Allows users to select an alert rule for removal Sends deletion request to the Alert Management Controller	Alert Management Controller

Table 6: CRC Card for Alert History

Class Name: Alert History (Boundary)	
Responsibility:	**Collaborators:**
Knows Alert Management Handles alert log storage	Alert Management

Table 7: CRC Card for Alert Rule Violation

Class Name: Alert Rule Violated (Boundary)	
Responsibility:	**Collaborators:**
Notifies the system when environmental data violates an alert rule Sends violation notifications to dashboards or user Sends violation information to the Alert Management Controller	Alert Management Controller

Table 8: CRC Card for Confirming an Alert Rule Violation

Class Name: Confirm Alert Rule Violated (Boundary)	
Responsibility:	**Collaborators:**
Allow city operators to confirm a detected alert violation Display the details of the alert event Send confirmation status to the Alert Management Controller	Alert Management Controller

Class Name: Main Dashboard	
Responsibility:	**Collaborators:**
Knows Main Dashboard Controller Handles display of main dashboard	Main Dashboard Controller

Class Name: API Controller	
Responsibility:	**Collaborators:**
Knows API Documentation Knows API DB Knows Main Dashboard Controller Facilitates communication and implementation of API calls	API Documentation API DB Main Dashboard Controller

Class Name: API Documentation	
Responsibility:	**Collaborators:**
Knows API Controller Responsible for displaying API documentation for developers	API Controller

Class Name: API Database	
Responsibility:	**Collaborators:**
Knows API Controller Knows Endpoints Knows active and inactive routes	API Controller

Class Name: Account Settings	
Responsibility:	**Collaborators:**

Class Name: Admin Dashboard Controller	
Responsibility:	**Collaborators:**
Knows Main Dashboard Controller Knows Admin Dashboard Knows Admin DB	Main Dashboard Admin Dashboard Admin DB

Class Name: View Account	
Responsibility:	**Collaborators:**

Class Name: Admin Dashboard	
Responsibility:	**Collaborators:**
Knows Admin Dashboard Controller Responsible for displaying administrator action interface	Admin Dashboard Controller

Class Name: Admin Database	
Responsibility:	**Collaborators:**
Knows Admin Dashboard Controller Knows Admin privileges Knows Admin action log history	Admin Dashboard Controller

Class Name: City Wide Dashboard Controller	
Responsibility:	**Collaborators:**
Knows City Wide Dashboard Knows Main Dashboard Controller Knows District Dashboard Controller	City Wide Dashboard Main Dashboard Controller District Dashboard Controller

Class Name: Account Management	
Responsibility:	**Collaborators:**

Class Name: City Dashboard	
Responsibility:	**Collaborators:**
Knows City Dashboard Controller Handles display of city dashboard	City Dashboard Controller

Class Name: Create Account	
Responsibility:	**Collaborators:**

Table 9: CRC Card for District Dashboard Controller

Class Name: District Dashboard Controller (Control)	
Responsibility:	**Collaborators:**
Knows City Wide Dashboard Controller Knows District Dashboard Knows Air Quality Dashboard Controller Knows Soil Quality Dashboard Controller Knows Temperature Dashboard Controller Knows Forest Health Dashboard Controller Coordinates navigation within the district dashboard Routes requests from the district dashboard to the correct environmental dashboard controller Aggregates district-level environmental information for presentation	City Wide Dashboard Controller District Dashboard Air Quality Dashboard Controller Soil Quality Dashboard Controller Temperature Dashboard Controller Forest Health Dashboard Controller

Class Name: Modify Account	
Responsibility:	**Collaborators:**

Table 10: CRC Card for District Dashboard

Class Name: District Dashboard (Boundary)	
Responsibility:	**Collaborators:**
Knows District Dashboard Controller Displays district-level environmental information Handles user selection of district environmental categories Sends user navigation requests to the District Dashboard Controller	District Dashboard Controller

Class Name: Login	
Responsibility:	**Collaborators:**

Table 11: CRC Card for Air Quality Dashboard Controller

Class Name: Air Quality Dashboard Controller (Control)	
Responsibility:	**Collaborators:**
Knows District Dashboard Controller Knows Air Quality Dashboard Knows Air Quality Database Retrieves air quality data for the selected district Processes air quality data for presentation Updates the Air Quality Dashboard with district-specific results	District Dashboard Controller Air Quality Dashboard Air Quality Database

Class Name: Account Settings	
Responsibility:	**Collaborators:**

Table 12: CRC Card for Air Quality Dashboard

Class Name: Air Quality Dashboard (Boundary)	
Responsibility:	**Collaborators:**
Knows Air Quality Dashboard Controller Displays district air quality information Handles user requests to view air quality metrics Presents processed air quality data to the user	Air Quality Dashboard Controller

Class Name: View Account	
Responsibility:	**Collaborators:**

Table 13: CRC Card for Air Quality Database

Class Name: Air Quality Database (Entity)	
Responsibility:	**Collaborators:**
Knows Air Quality Dashboard Controller Stores district air quality data Provides requested air quality records Maintains air quality measurement history	Air Quality Dashboard Controller

Class Name: Account Database	
Responsibility:	**Collaborators:**

Table 14: CRC Card for Soil Quality Dashboard Controller

Class Name: Soil Quality Dashboard Controller (Control)	
Responsibility:	**Collaborators:**
Knows District Dashboard Controller Knows Soil Quality Dashboard Knows Soil Quality Database Retrieves soil quality data for the selected district Processes soil quality data for presentation Updates the Soil Quality Dashboard with district-specific results	District Dashboard Controller Soil Quality Dashboard Soil Quality Database

Table 15: CRC Card for Soil Quality Dashboard

Class Name: Soil Quality Dashboard (Boundary)	
Responsibility:	**Collaborators:**
Knows Soil Quality Dashboard Controller Displays district soil quality information Handles user requests to view soil quality metrics Presents processed soil quality data to the user	Soil Quality Dashboard Controller

Table 16: CRC Card for Soil Quality Database

Class Name: Soil Quality Database (Entity)	
Responsibility:	**Collaborators:**
Knows Soil Quality Dashboard Controller Stores district soil quality data Provides requested soil quality records Maintains soil quality measurement history	Soil Quality Dashboard Controller

Table 17: CRC Card for Temperature Dashboard Controller

Class Name: Temperature Dashboard Controller (Control)	
Responsibility:	**Collaborators:**
Knows District Dashboard Controller Knows Temperature Dashboard Knows Temperature Database Retrieves temperature data for the selected district Processes temperature data for presentation Updates the Temperature Dashboard with district-specific results	District Dashboard Controller Temperature Dashboard Temperature Database

Table 18: CRC Card for Temperature Dashboard

Class Name: Temperature Dashboard (Boundary)	
Responsibility:	**Collaborators:**
Knows Temperature Dashboard Controller Displays district temperature information Handles user requests to view temperature metrics Presents processed temperature data to the user	Temperature Dashboard Controller

Table 19: CRC Card for Temperature Database

Class Name: Temperature Database (Entity)	
Responsibility:	**Collaborators:**
Knows Temperature Dashboard Controller Stores district temperature data Provides requested temperature records Maintains temperature measurement history	Temperature Dashboard Controller

Table 20: CRC Card for Forest Health Dashboard Controller

Class Name: Forest Health Dashboard Controller (Control)	
Responsibility:	**Collaborators:**
Knows District Dashboard Controller Knows Forest Health Dashboard Knows Forest Health Database Retrieves forest health data for the selected district Processes forest health data for presentation Updates the Forest Health Dashboard with district-specific results	District Dashboard Controller Forest Health Dashboard Forest Health Database

Table 21: CRC Card for Forest Health Dashboard

Class Name: Forest Health Dashboard (Boundary)	
Responsibility:	**Collaborators:**
Knows Forest Health Dashboard Controller Displays district forest health information Handles user requests to view forest health metrics Presents processed forest health data to the user	Forest Health Dashboard Controller

Table 22: CRC Card for Forest Health Database

Class Name: Forest Health Database (Entity)	
Responsibility:	**Collaborators:**
Knows Forest Health Dashboard Controller Stores district forest health data Provides requested forest health records Maintains forest health measurement history	Forest Health Dashboard Controller

Class Name: Modify Account	
Responsibility:	**Collaborators:**

A Division of Labour

Benjamin Bloomfield

- 3.1: Justification of considered architectures that were not chosen.
- 3.2.1 Subsystem descriptions.
- 4. CRC Cards:
  - Alert Management Controller

  - Create Alert Rule
  - Modify Alert Rule
  - Remove Alert Rule
  - Alert Rule Violated
  - Confirm Alert Rule Violated

Date: March 7th, 2025

Vikram Chandar

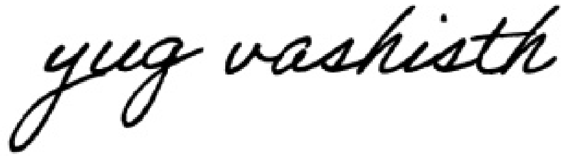
-

Trisha Panchal

-

Yug Vashisth

- Section 3.1: Structural architecture diagram
- Section 4. CRC Cards:
  - District Dashboard Controller
  - District Dashboard
  - Air Quality Dashboard Controller
  - Air Quality Dashboard
  - Air Quality DB
  - Soil Quality Dashboard Controller
  - Soil Quality Dashboard
  - Soil Quality DB
  - Temperature Dashboard Controller
  - Temperature Dashboard
  - Temperature DB
  - Forest Health Dashboard Controller
  - Forest Health Dashboard
  - Forest Health DB



Date: March 7th, 2025

Hamza Nadeem

- Section 1
- Created cards for each class for Section 4
- 4. CRC Cards:
  - Alert Database
  - Alert History
  - Main Dashboard
  - City Wide Dashboard Controller
  - City Wide Dashboard

Date: March 7th, 2025